

Module 7
Attacking Machine Learning Models

Can you attack a model?

Can you attack a model?

YES!

Attack Types

- Model Evasion
- Poisoning
- Membership Inference
- Model Extraction / Inversion



White box attacks require access to the model, and black box only require access to the output.

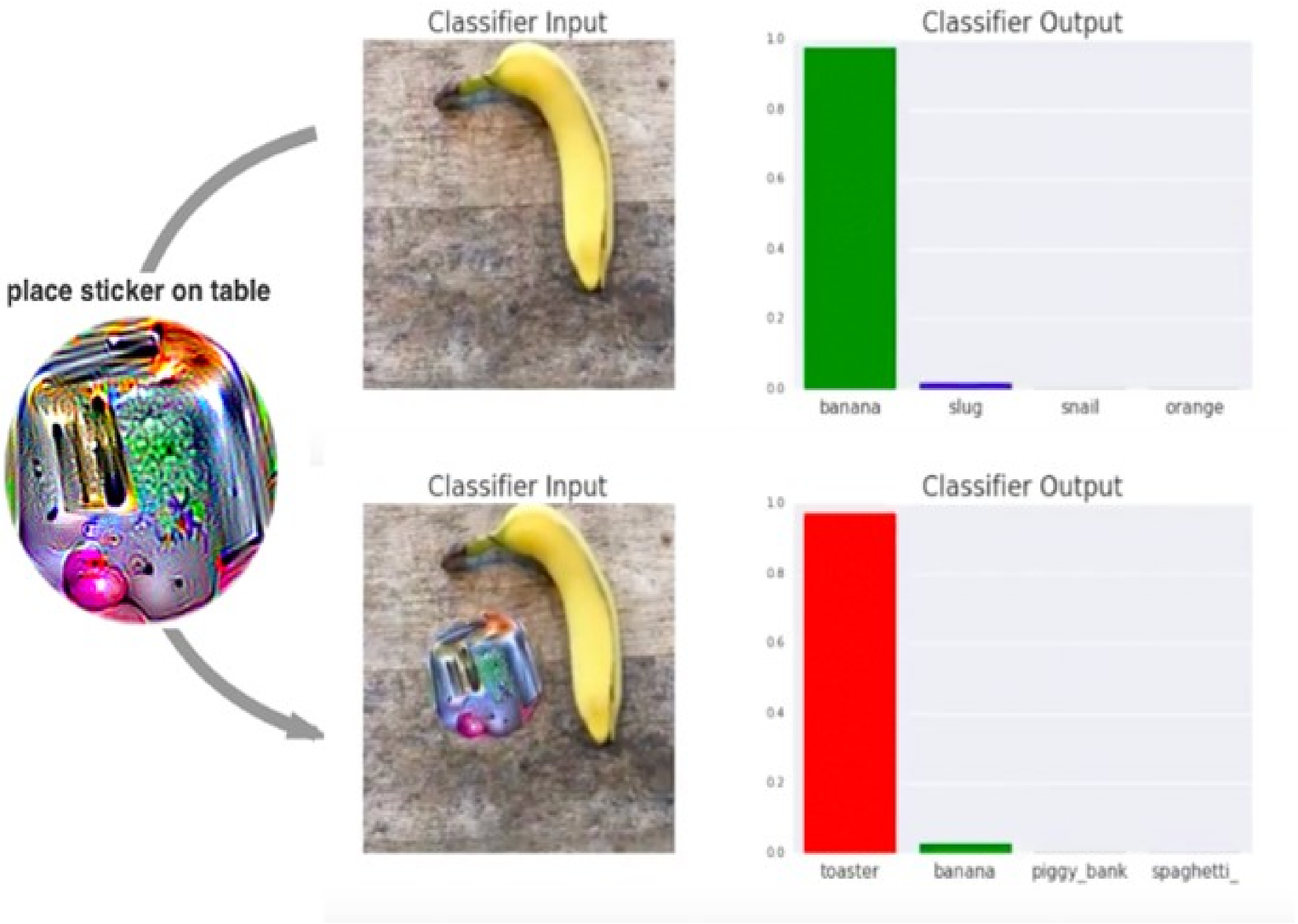
Attack Surface

Phase	Description	Activity Type
Data Collection	Models require data to be trained. Data is usually collected from both public and platform sources with a specific model in mind. This is an ongoing process and data will continue to be collected from these sources.	Train
Data Processing	The collected data is processed in any number of ways before being introduced to an algorithm for both training and inference.	Train
Model Training	The processed data is then ingested by an algorithm and a model is trained.	Train
Model Validation	After a model is trained, it is validated to ensure accuracy, robustness, explainability, or any number of other metrics.	Train
Model Deployment	The trained model is embedded in a system for use in production. Machine learning is deployed in a wide variety of ways – inside autonomous vehicles, on a web API, in client-side applications.	Inference
System Monitoring	Once the model has been deployed, the “system” is monitored. This includes aspects of the system that may not related to the ML model directly.	Inference

Attack Types

Attack	Description	Activity Type
Functional Extraction	An adversary successfully copies a model functionality.	Inference
Model Evasion	An adversary successfully causes a model to misclassify an input.	Inference
Model Inversion	An adversary successfully recovers internal state information from the model such as trained weights.	Inference
Membership Inference	An adversary successfully recovers data the model was trained on.	Inference
Model Poisoning	An adversary successfully alters inputs to the model at train time.	Train
System Compromise	Traditional attacks against infrastructure components.	N/A

Model Evasion



https://gizmodo.com/this-simple-sticker-can-trick-neural-networks-into-thin-1821735479

An attack caused a model to label this image as a 45mph Speed Limit Sign



=



Ivan Evtimov et al. ."Robust Physical-World Attacks on Deep Learning Models" (2017)

An attack caused a model to label this image as a Stop Sign



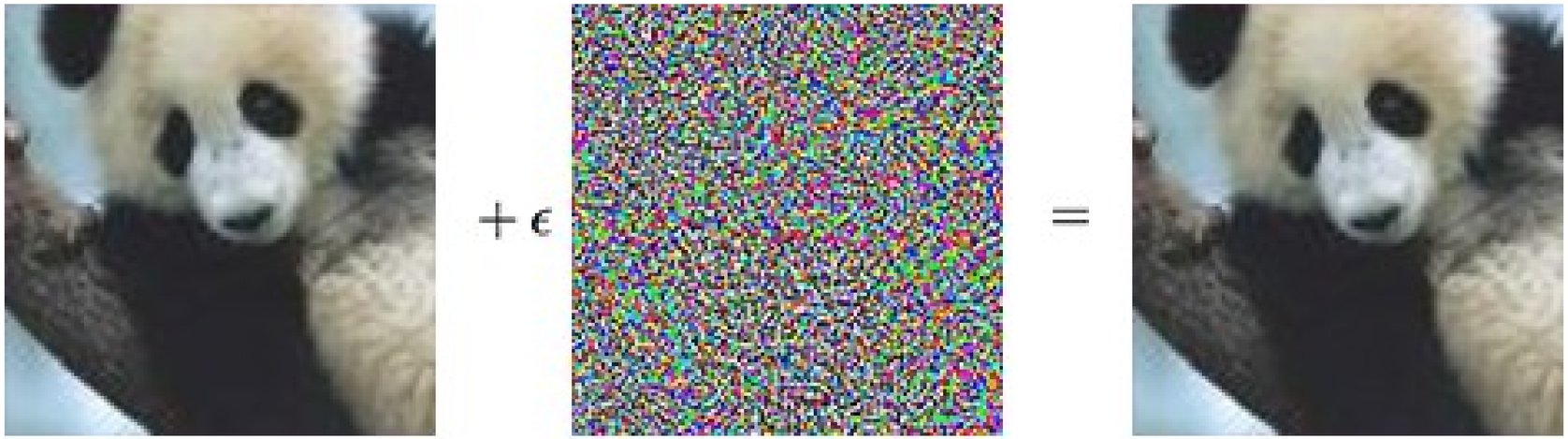
=



Ivan Evtimov et al. ."Robust Physical-World Attacks on Deep Learning Models" (2017)

Altering a Prediction

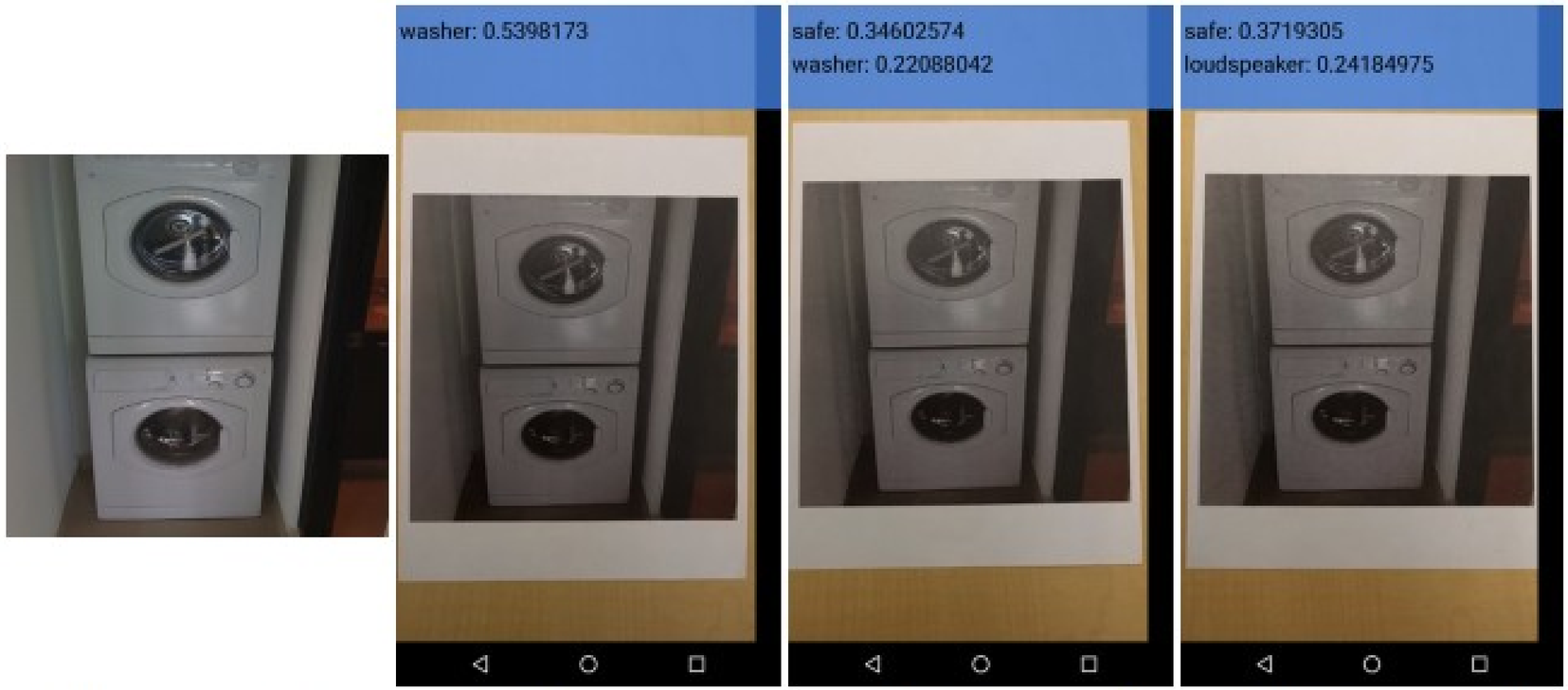
By adding small perturbations to an image, it is possible to completely alter the prediction.



Explaining and Harnessing Adversarial Examples, Goodfellow et al., 2014

Altering a Prediction

Photos taken on a smartphone and printed out can be altered in this way.



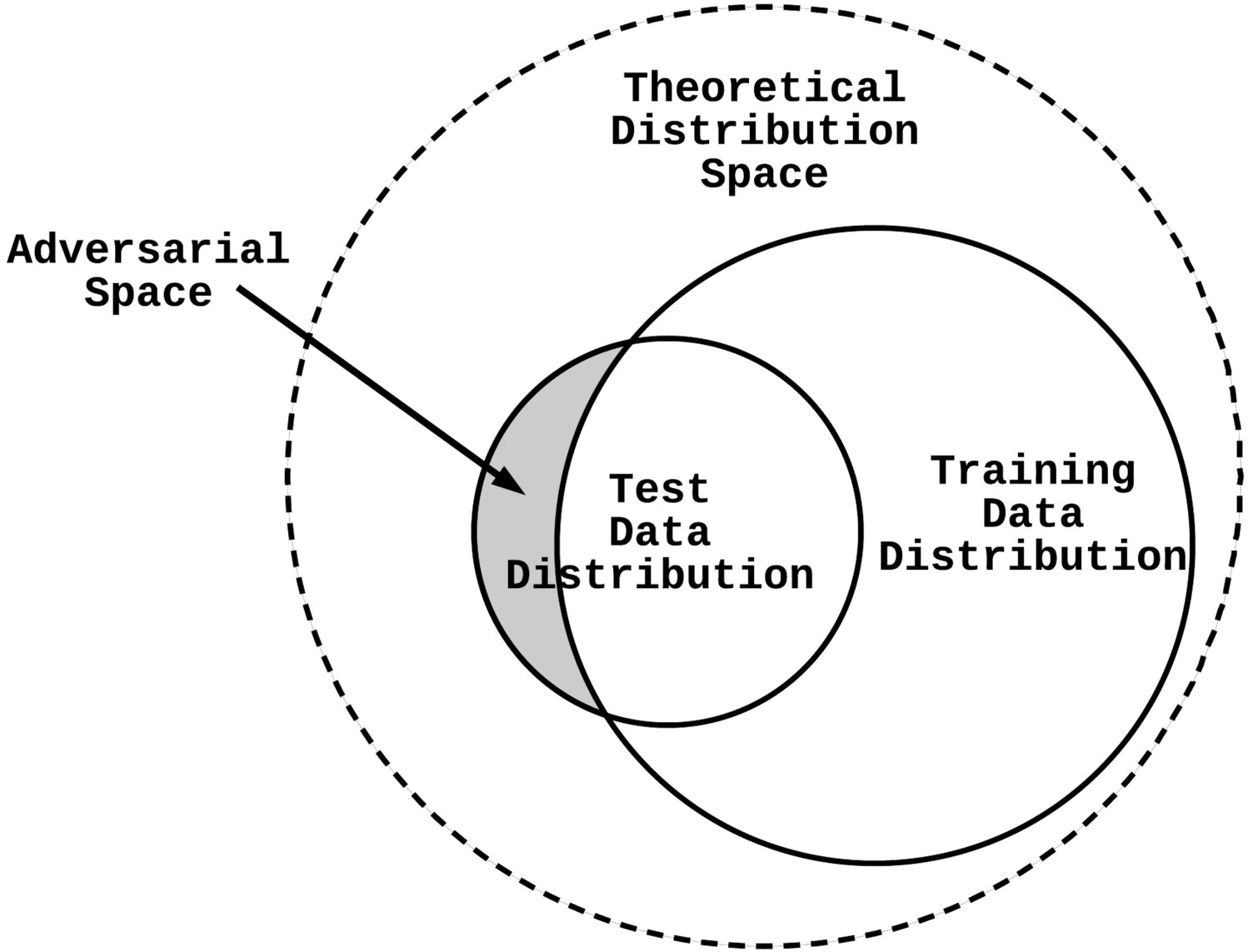
(a) Image from dataset (b) Clean image (c) Adv. image, $\epsilon = 4$ (d) Adv. image, $\epsilon = 8$

Explaining and Harnessing Adversarial Examples, Goodfellow et al., 2014

Model Evasion Attacks

- A model evasion attack is when a model is fed a carefully crafted input, known as an adversarial example, that is designed to be misclassified.
- This attack was first described in 2004.
- Model evasion attacks can be successfully executed either as a black or white box attack.
- The model's complexity is not a defense against an evasion attack.

https://towardsdatascience.com/evasion-attacks-on-machine-learning-or-adversarial-examples-



Clarence Chio, David Freeman. Machine Learning & Security. Pg. 319 (2018)

Functional Extraction

Functional Extraction



Prarie Rattlesnake



Bull Snake

http://poofree.com/colorado-rattlesnakes-and-your-furry-friend-duty-free-pets-colorado-pooper-scooper/rattlesnake-vs-bull-snake-identify-pooper-scooper-boulder-colorado/

Functional Extraction: So What?

- Having a “copy” of a model to play with allows attackers to better attempt other attack methods
  - Evasion - Attackers can develop tactics to evade the defensive models using their copy
  - Poisoning - Attackers can practice poisoning their model and establish some good baseline idea of what it would take to move boundaries on the target model

http://poofree.com/colorado-rattlesnakes-and-your-furry-friend-duty-free-pets-colorado-pooper-scooper/rattlesnake-vs-bull-snake-identify-pooper-scooper-boulder-colorado/

Model Inversion
&
Membership Inference

Inversion/Inference Example

- A facial recognition model “ W “ is produced using a training procedure “ Y “ and data “ X “.
- Model is released to the public for use (white box)
- Given the model and possible output labels can training data be recovered?

Inversion/Inference Example



Figure 1: An image recovered using a new model inversion attack (left) and a training set image of the victim (right). The attacker is given only the person's name and access to a facial recognition system that returns a class confidence score.

https://www.cs.toronto.edu/~toni/Courses/Fairness/Lectures/ML-and-DP-v2.pdf

Inversion/Inference So What

- Sensitive Information
  - Credit Card Numbers
  - Addresses
  - Survey data

risk-taking behaviors [17]. To support the analysis, FiveThirtyEight commissioned a survey of 553 individuals from SurveyMonkey, which collected responses to questions such as: "Do you ever smoke cigarettes?", "Have you ever cheated on your significant other?", and of course, "How do you like your steak prepared?". Demographic characteristics such as

and 11 variables, including basic demographic information and responses to questions such as, "How happy are you in your marriage?" and "Have you watched X-rated movies in the last year?" We discarded rows that did not contain re-

https://www.cs.toronto.edu/~toni/Courses/Fairness/Lectures/ML-and-DP-v2.pdf

Poisoning

Defined

- **Poisoning Attack:** Used with online learning systems. Injecting data to cause a model to modify its decision boundary in a particular direction.

Poisoning Attack

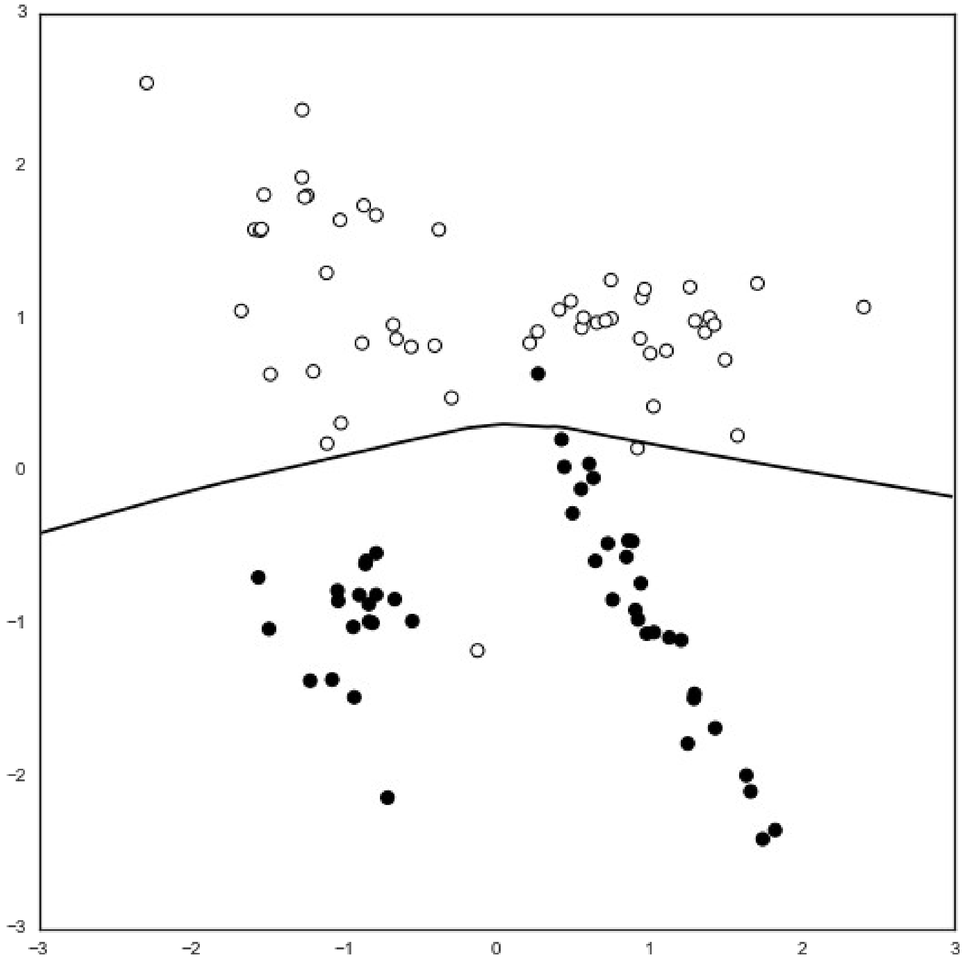
- Online learning systems automatically adjust model parameters over time based on input
- Poisoning attacks, an actor injects new data into a retraining set with the intent of altering the decision boundaries.

Poisoning Attack



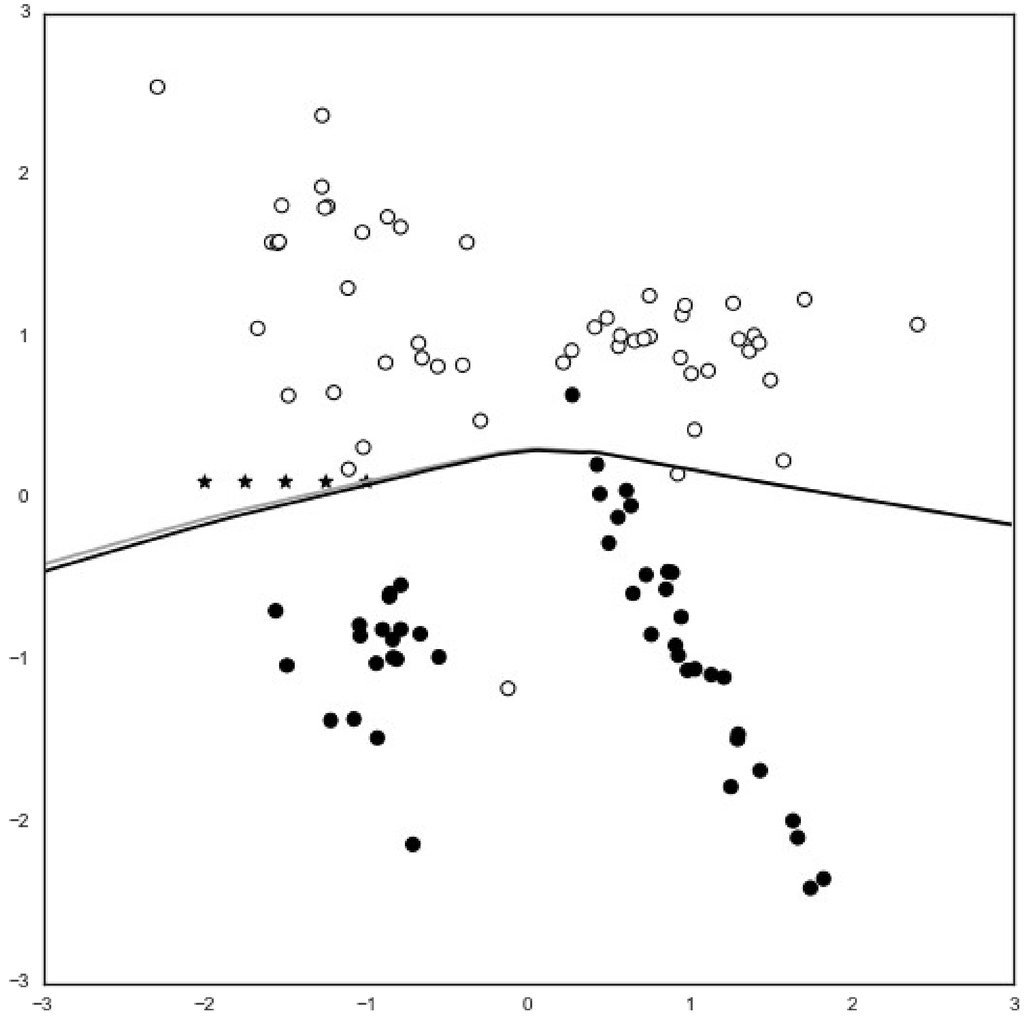
Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



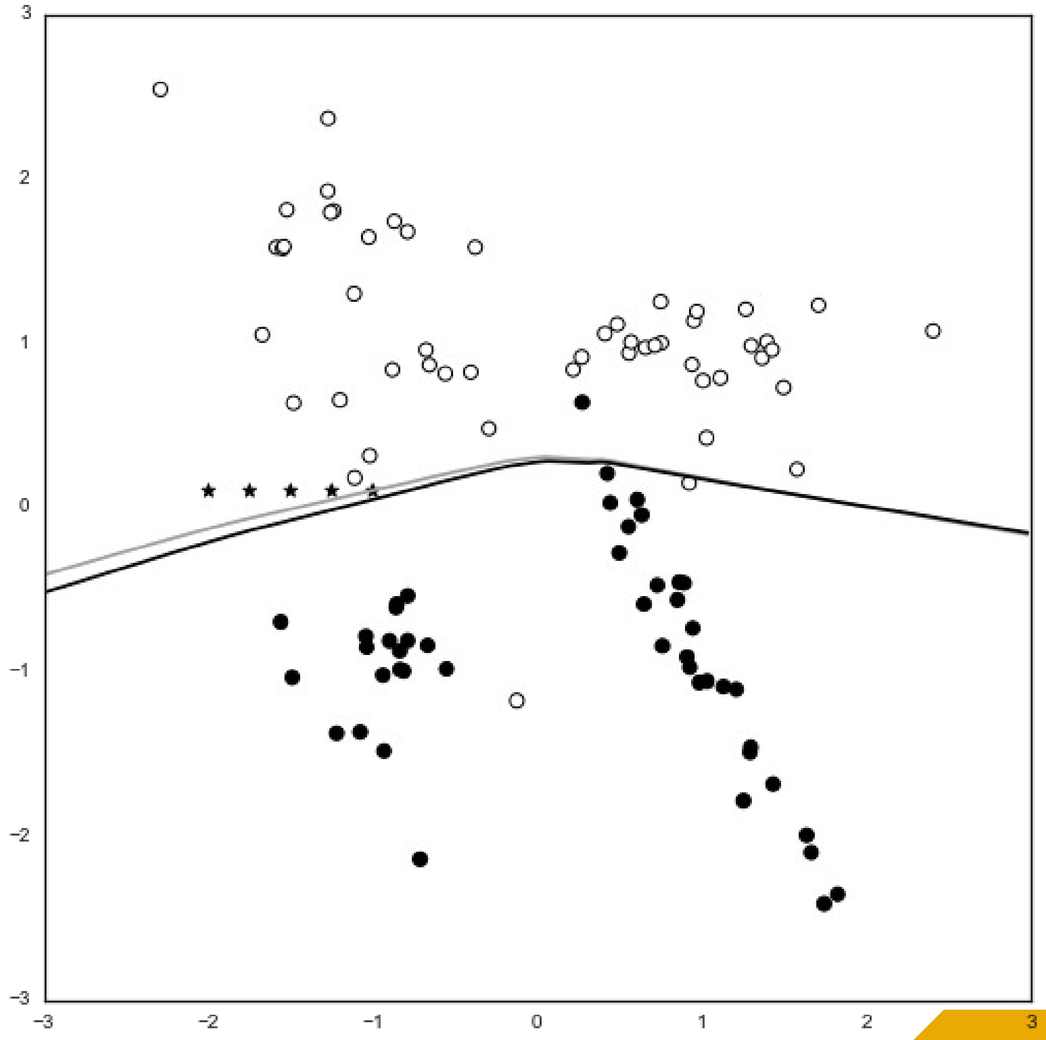
Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



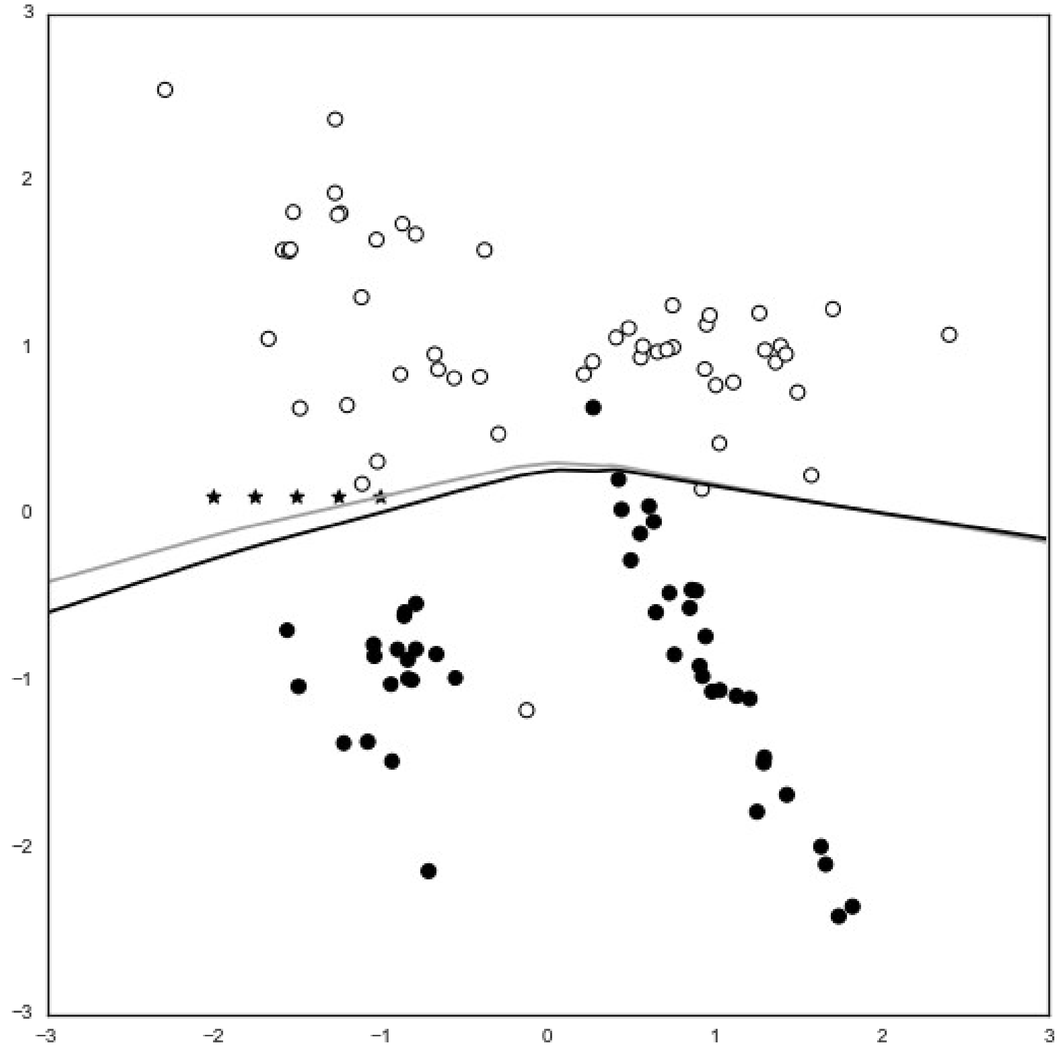
Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



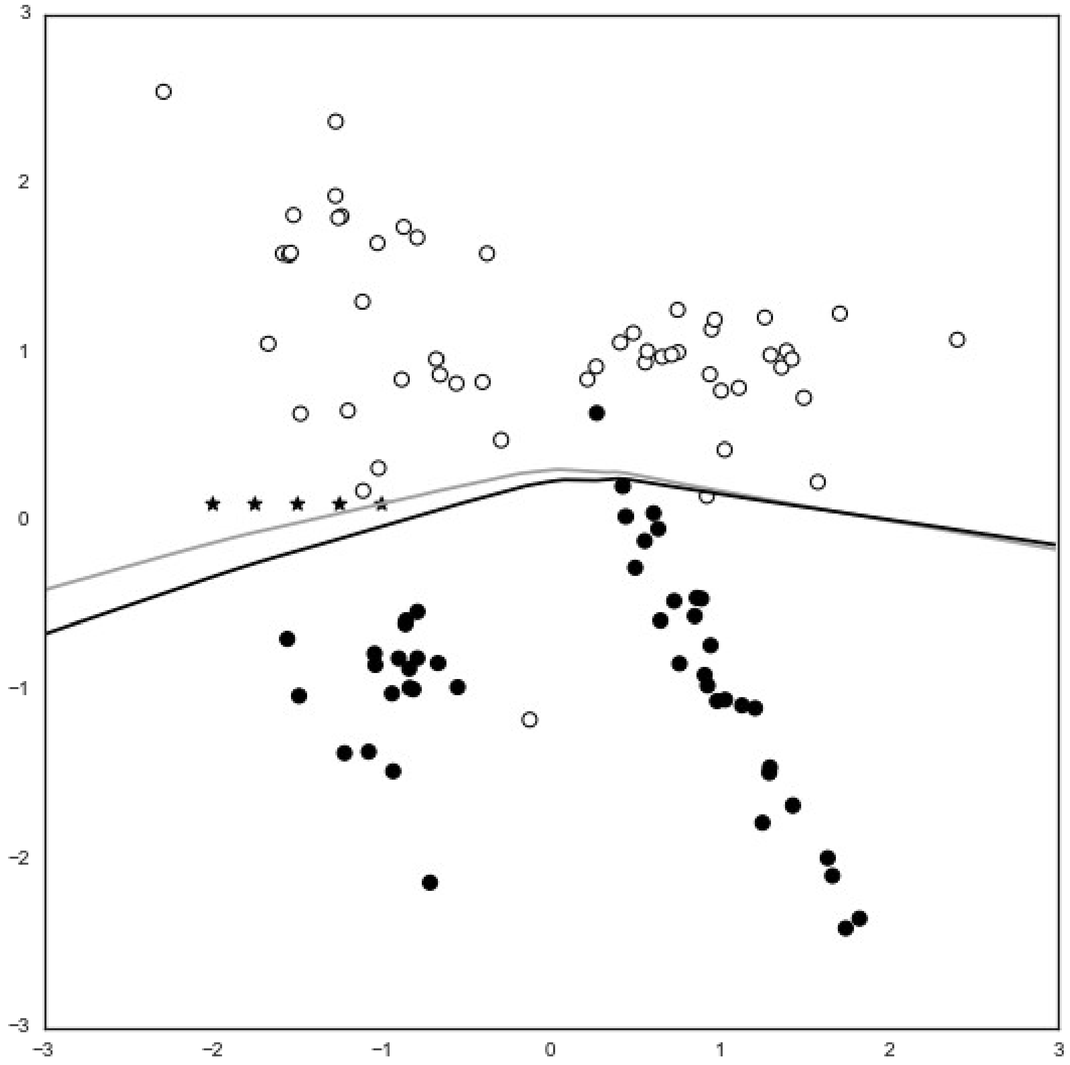
Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



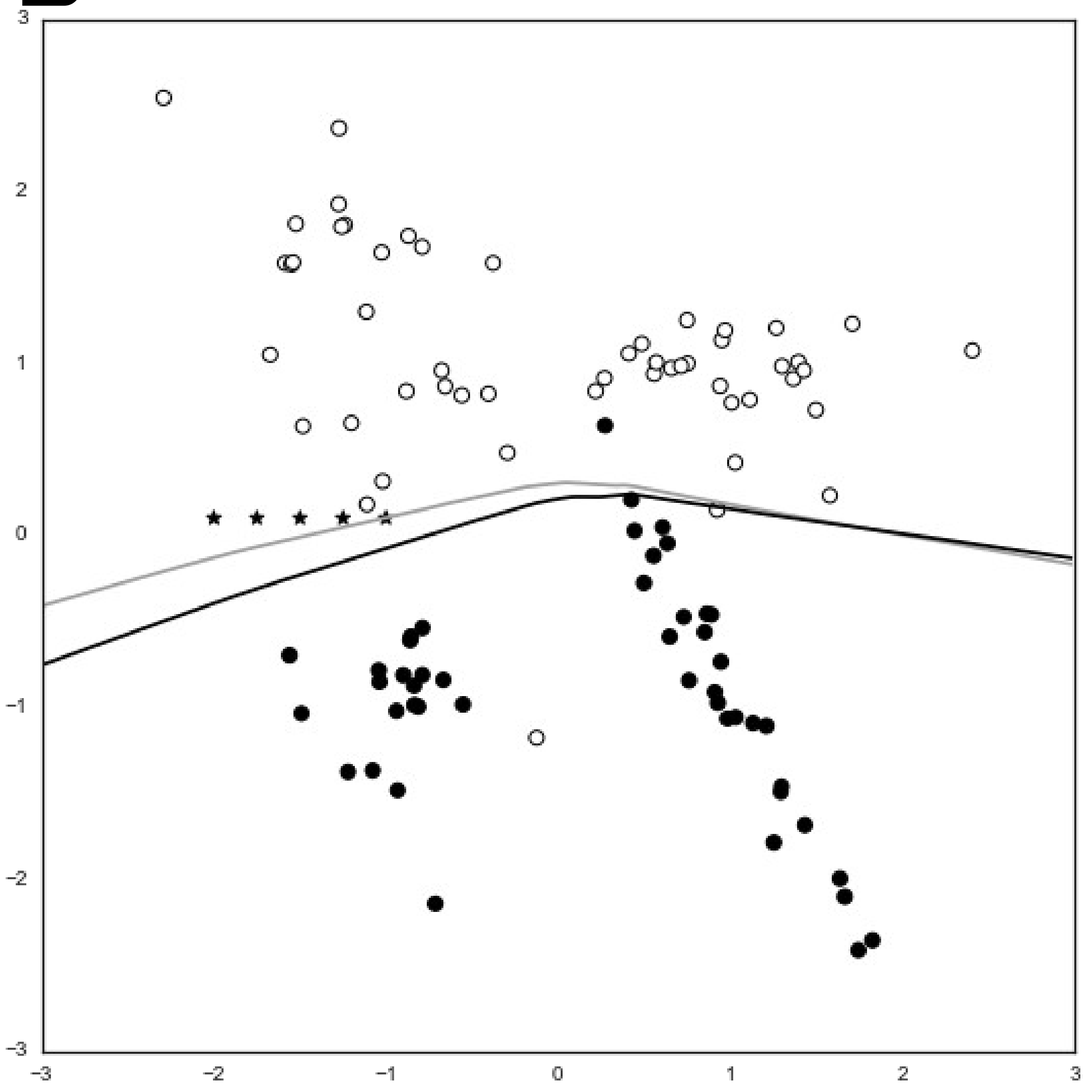
Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attack



Clarence Chio, David Freeman. Machine Learning & Security. Pg. 322 (2018)

Poisoning Attacks

- Require access to either the predictions or the probabilities for an effective attack
- Longer periods between retraining
- Periodically analyzing retraining data to detect "boiling frog" attacks
- Avoiding real time online learning systems unless absolutely necessary

Additional Readings

- Alexey Kurakin et al. "Adversarial Examples in the Physical World" (2016)
- Anish Athalye et al. "Synthesizing Robust Adversarial Examples" (2017)
- Ivan Evtimov et al. "Robust Physical World Attacks on Machine Learning Models" (2017)
- Weilin Xu et al. "Automatically Evading Classifiers: A Case Study on PDF Malware Classifiers" (2016)

Questions?

The Details...

Model Evasion

Model Evasion Methods

- Gradient Based Attacks (White box)
- Score Based (Black Box / Score based)
- Decision-Based (Only class labels)
- Physical World Attacks

Gradient Based Attacks

Gradient Based Attacks

- Neural networks learn by adjusting weights in the direction that reduces error. This direction is given by the gradient of the loss function. A gradient tells us which way and how strongly the output will change if we nudge the input.
- An attacker can hijack this same mechanism. Instead of using gradients to make the model more accurate (as in training), they use gradients to find how to slightly alter the input so the model makes a mistake.
- Gradient based attacks can be used against neural networks, logistic regression and other linear classifiers and SVMs. They will not work against tree-based models.

Gradient Based Methods

Think of the model's decision boundary (the "line" or "surface" that separates one class from another).

- In many regions, the boundary is surprisingly close to real data points.
- The gradient points directly toward that boundary.
- By following this direction and adding a tiny perturbation, the input can be shifted just enough to cross into the wrong class.

From a human perspective, the input looks unchanged — but to the model, it has jumped to the other side of the boundary.

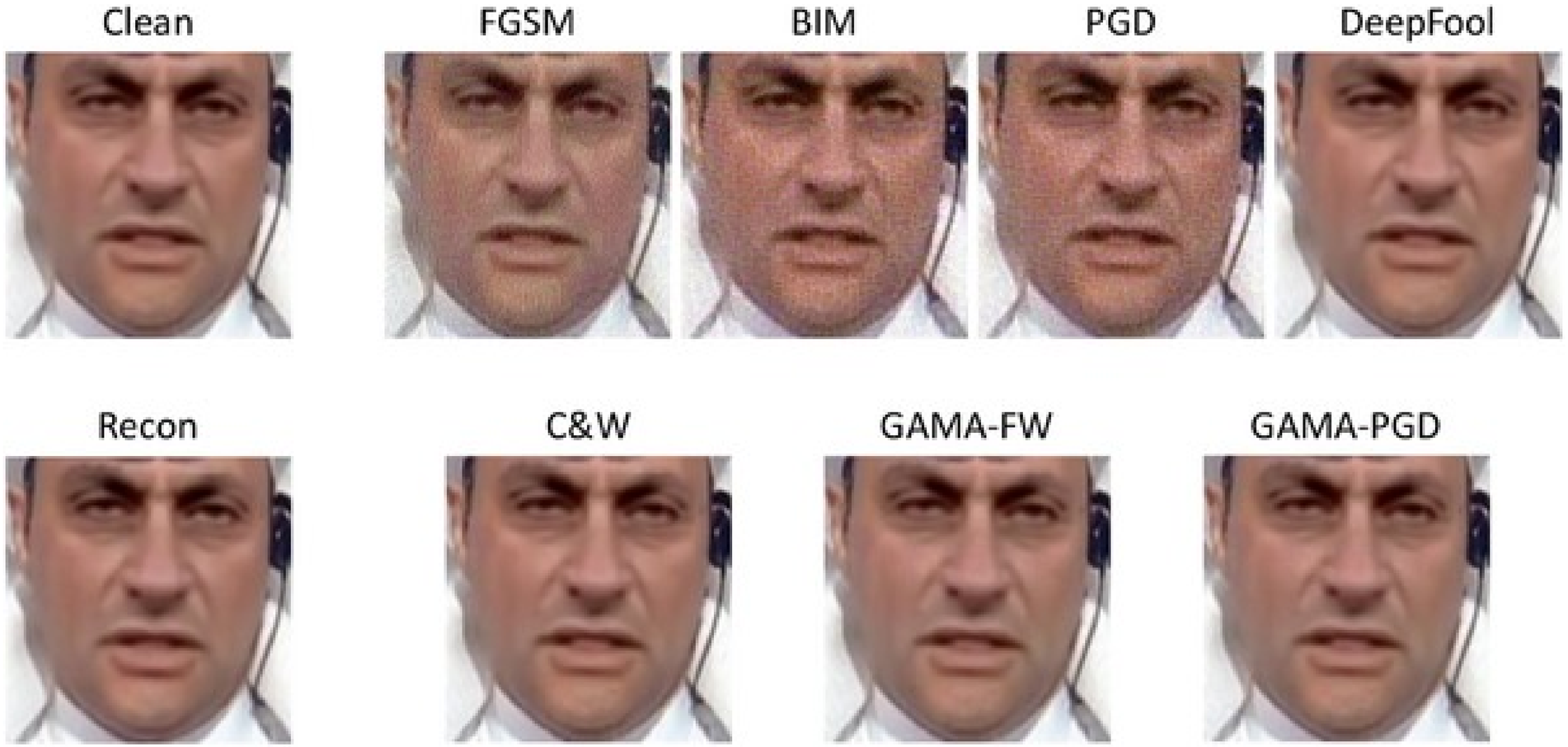
Gradient Based Methods - Crafting the Attack

1. Compute gradient direction: Determine which small changes to the input would most increase the model's error.
2. Perturb the input: Add a carefully chosen "noise" pattern that follows this direction.
3. Keep it subtle: The perturbation is constrained to be very small so it remains invisible (for images) or imperceptible (for structured/text data).
4. Test the model: The perturbed input is fed back to the model, which now misclassifies with high confidence.

Gradient Based Methods - Crafting the Attack

- **Single-step (FGSM)**: One quick jump in the vulnerable direction, fast but less stealthy.
- **Iterative (Projected Gradient Descent)**: Multiple small steps with projection back into the valid region — stronger, harder to defend against.
- **Optimized (Carlini & Wagner)**: Searches for the smallest possible perturbation that still flips the classification — extremely subtle.

Gradient Based Methods



https://www.researchgate.net/figure/Illustration-of-adversarial-examples-generated-by-FGSM-PGD-BIM-DeepFool-C-W-GAMA-FW_fig2_375277898

Gradient Based Methods

- Neural networks often rely on narrow statistical cues rather than robust features.
- These cues are spread across thousands of input dimensions. A tiny change in each dimension, when combined, can push the model over the edge.
- Humans don’t notice these changes because our perception isn’t tuned to those specific statistical quirks.

Fast Gradient Sign Method (FSGM)

- FSGM is the simplest and fastest gradient based attack.
- Instead of carefully optimizing, it takes one big step in the direction a model is most vulnerable.

FSGM: How It Works

- Look at the gradient: The attacker computes how sensitive the model's prediction is to changes in each input feature.
- Choose a direction: For each feature (like a pixel in an image), decide whether increasing or decreasing its value will push the model toward a mistake.
- Make the step: Nudge every feature slightly in that "bad" direction.
- Result: The modified input looks almost identical to the original but causes the model to misclassify.

FSGM: Pros & Cons

Advantages of FSGM

- Speed: It’s very fast — just one calculation of the gradient.
- Effectiveness: Even such a crude attack often works surprisingly well.

Drawbacks

- It’s widely used as an introductory attack and as a benchmark for defenses.
- Because it’s a single-step, it’s easier to defend against compared to iterative methods like PGD.
- As with all gradient based attacks, it is a white box attack.

Projected Gradient Descent (PGD)

PGD is basically FGSM repeated many times, with guardrails.

- Instead of one big push (FGSM), PGD takes many small steps in the gradient direction.
- After each step, it “projects” the perturbed input back into a valid region (like a ball around the original input).
- This ensures the adversarial example is powerful but still subtle.

Projected Gradient Descent (PGD)

PGD is basically FGSM repeated many times, with guardrails.

- Instead of one big push (FGSM), PGD takes many small steps in the gradient direction.
- After each step, it “projects” the perturbed input back into a valid region (like a ball around the original input).
- This ensures the adversarial example is powerful but still subtle.

Projected Gradient Descent (PGD) - How It Works

1. Start with the original input.
2. Compute the gradient (the most sensitive direction).
3. Take a small step in that direction.
4. Check the constraint: if the perturbation is too large, pull it back inside the allowed range.
5. Repeat steps 2–4 many times.
6. End with an adversarial input that is almost indistinguishable to humans but very likely to fool the model.

Projected Gradient Descent (PGD) - Pros & Cons

Advantages:

- Much stronger than FGSM: Iterations allow the attack to explore the space more effectively.
- Robust benchmark: PGD is often called the “gold standard” for testing model robustness.
- Harder to defend: Defenses that stop FGSM usually fail against PGD.
- Flexible: Works with different types of constraints (pixel range, L_2 distance, etc.).

Disadvantages:

- PGD is MUCH SLOWER that FSGM
- Requires thousands of queries in black-box settings
- Detectable

Carlini & Wagner (C&W) Attack

- The C&W attack is an optimization-driven adversarial attack.
- Instead of taking one step (FGSM) or many steps (PGD), it solves an optimization problem:“What is the smallest possible change I can make to flip the model’s decision?”
- C&W is extremely stealthy and precise.

Carlini & Wagner (C&W) Attack: How it Works

1. Start with the original input.
2. Search for the tiniest perturbation that makes the model misclassify.
3. Use optimization techniques to balance two goals:
4. Keep the perturbation as small as possible.
5. Ensure the model actually predicts the wrong label.
6. Iteratively refine the adversarial example until both goals are satisfied.

Carlini & Wagner (C&W) Attack: Pros & Cons

Advantages:

- **Extremely subtle**: Perturbations are often imperceptible to humans, even when magnified.
- **Highly effective**: Bypasses many defenses that can stop simpler attacks (like FGSM or basic PGD).
- **Targeted flexibility**: Can be tuned to force a model to misclassify into a specific class (not just any wrong class).
- **Benchmark standard**: Widely used in academia and industry as a “stress test” for proposed defenses.
- **Robust across domains**: Works on images, audio, text embeddings, and other differentiable inputs.

Disadvantages:

- **Computationally expensive**: Requires iterative optimization, making it slower than FGSM or PGD.
- **Requires white-box access**: Needs model gradients and logits, so not directly usable in pure black-box settings.
- **Parameter tuning:** Success depends heavily on choosing the right optimization hyperparameters (step size, regularization strength).
- **Not always practical in real-time**: Too slow for time-sensitive attacks against deployed systems.

Score Based Attacks

Score Based Attacks

- Score-based attacks apply when an attacker doesn’t have full access to the model’s parameters (no gradients), but does see output probabilities or confidence scores.
- Instead of using gradients directly, the attacker estimates them by probing the model:
  - Make tiny changes to the input.
  - Watch how the output scores shift.
  - Piece together an approximate gradient from these changes.

Score Based Attacks: How They Work

1. Query the model with an input.
2. Slightly perturb features (like a pixel, word, or tabular field).
3. Record the output scores for each perturbation.
4. Infer sensitivity and identify which features cause the biggest change in output.
5. Iteratively craft adversarial input using this estimated gradient.

Score Based Attacks: Methods

Zeroth-Order Optimization (ZOO):

- Estimates gradients with finite differences (probe each feature one at a time).
- Accurate, but very query-intensive (slow for high-dimensional inputs).

Natural Evolution Strategies (NES):

- Smarter method that samples groups of perturbations from random directions.
- Aggregates results to approximate the gradient with fewer queries.
- Much more efficient for real-world API attacks.

Score Based Attacks: Pros & Cons

Pros

- Works in black-box settings (no access to model internals).
- Effective against commercial APIs that return probabilities (e.g., image recognition, sentiment analysis).
- Flexible: Can be applied across domains (images, text, malware).
- Closer to real-world attacker capabilities than pure white-box gradient attacks.

Cons

- Query expensive: ZOO in particular can require thousands of API calls.
- Slow: Not practical for real-time attacks unless optimized.
- Easier to detect: Many queries may trigger API rate-limits, logging, or anomaly detection.
- Weaker against models that hide scores: If only class labels are returned, score-based methods fail.

Zeroth Order Optimization (ZOO)

- ZOO is a score-based adversarial attack. It’s designed for black-box settings where the attacker:
  - Cannot see the model’s architecture or parameters.
  - Cannot compute gradients directly.
  - Can query the model and receive confidence scores or probabilities for predictions.
- The challenge: how to run an attack like FGSM or PGD when you don’t have gradients?

Zeroth Order Optimization (ZOO)

- ZOO reconstructs gradients indirectly by probing the model's outputs.
  1. Take the original input.
  2. Nudge one feature slightly (say, brighten a single pixel in an image).
  3. Compare the new output scores to the original.
  4. The amount and direction of change tells you how sensitive the model is to that feature.
- By repeating this process for many features, ZOO builds up an approximate gradient vector without ever seeing the true internals.

Zeroth Order Optimization (ZOO): How it works

1. Pick a feature (pixel, word embedding dimension, tabular value).
2. Apply a tiny perturbation and query the model again.
3. Record the score change — did it help push the model closer to a wrong class?
4. Do this for every feature (or a subset, if input space is huge).
5. Combine the results into an estimated gradient that can then drive an adversarial attack (just like FGSM or PGD would).

Zeroth Order Optimization (ZOO):

Pros:

- **Black-box ready**: Needs only query access, not model internals.
- **Generalizable**: Works on any model type, as long as it returns scores.
- **Flexible**: Attack formulation is similar to gradient-based ones once the estimate is built.

Cons:

- **Extremely query-heavy**: For high-dimensional inputs like images, probing every feature is prohibitively expensive (millions of queries).
- **Slow**: Iterative optimization can take orders of magnitude longer than white-box attacks.
- **Detectable**: High query volume can trigger alarms or rate limits in real systems.
- **Scales poorly**: Not practical for large-scale attacks against modern deep networks without heavy optimization.

Natural Evolution Strategies (NES)

- NES is a score-based black-box attack like ZOO, but it’s designed to be much more efficient.
- Instead of probing every feature one by one (which is painfully slow for images or text), NES samples entire directions in the input space.
- By analyzing how the model’s output scores shift across these sampled directions, NES reconstructs a useful approximation of the gradient with far fewer queries.

Natural Evolution Strategies (NES)

NES borrows from evolutionary algorithms:

- Imagine generating a “population” of random perturbations (noise vectors).
- Apply them to the input and query the model.
- Perturbations that make the model more likely to misclassify are considered “better.”
- By aggregating the results, you infer the direction in input space that consistently reduces the model’s accuracy.

This aggregated direction functions like a gradient estimate, guiding the attack.

Natural Evolution Strategies (NES): How it works

1. Start with a clean input.
2. Sample a batch of random noise vectors from a distribution (like Gaussian noise).
3. Perturb the input with each noise vector and query the model.
4. Record how the scores shift for each perturbation.
5. Aggregate results: perturbations that consistently push scores toward misclassification reveal the approximate gradient direction.
6. Update the input in that direction, repeating the process until the model is fooled.

Natural Evolution Strategies (NES): Pros & Cons

Pros:

- Much more efficient than ZOO: Requires far fewer queries.
- Scales to high dimensions: Works even on large images where ZOO is infeasible.
- Black-box friendly: Needs only score access, not gradients.
- Well-suited for API attacks: Can operate under query budgets that would make ZOO impossible.

Cons:

- Still slower than white-box methods: Multiple queries per update are required.
- Less precise than true gradients: Estimations can be noisy, especially with small query budgets.
- Requires score access: If the API only gives class labels, NES doesn't work (you'd need decision-based methods instead).

Decision Based Methods

Decision Based Methods

Since gradients and scores are unavailable, the attacker instead focuses on finding the decision boundary itself.

- Start with an input that is already misclassified (an adversarial-looking example).
- Iteratively move closer to the original input, shrinking the perturbation, while ensuring the sample still crosses the decision boundary.
- Over time, this process converges on a minimal adversarial perturbation even though the attacker never saw gradients or scores.

Decision Based Attacks

Boundary Attack

- Start with a heavily perturbed adversarial input (e.g., pure noise classified incorrectly).
- Gradually reduce the perturbation while checking if the model still misclassifies.
- Slow but conceptually simple: a kind of “walk along the boundary.”

HopSkipJump (HSJ) Attack

- More efficient version of Boundary Attack.
- Uses binary search between adversarial and original samples to quickly hone in on the decision boundary.
- Then estimates the boundary’s local direction with random sampling, guiding the attack more intelligently.
- Requires fewer queries than Boundary Attack, making it more practical.

Decision Based Attacks

- **Most restrictive setting**: Work even when the defender hides probabilities.
- **Powerful against hardened APIs**: Many commercial systems only return class labels, thinking that hides vulnerability.
- **Demonstrates robustness gaps**: Even with minimal information, models can often still be fooled.

Decision Based Attacks

Pros:

- True black-box: Needs nothing but predicted labels.
- Applicable to real-world APIs that reveal no confidence scores.
- Can produce very subtle adversarial examples despite limited feedback.

Cons

- Very query-intensive: Can take thousands of queries to converge.
- Slower than gradient or score-based methods.
- Not always practical in time-sensitive or rate-limited environments.
- Noisy estimation: Without gradients or scores, attacks are less precise

Boundary Attack

Boundary Attack

The Boundary Attack is a decision-based adversarial method.

- It assumes the attacker only has access to final labels (no scores, no gradients).
- The attack begins with an input that is already misclassified (for example, a noise image that the model doesn’t recognize correctly).
- From there, it walks back toward the original input, making small reductions in perturbation while keeping the input misclassified.
- Over many steps, the attack converges on an adversarial example that looks close to the original but still fools the model.

Boundary Attack: How it Works

1. **Initialization**: Find any input the model misclassifies. This might look nothing like the original data.
2. **Iterative refinement**:
   - Move the input slightly closer to the original sample.
   - After each move, check if the model still misclassifies it.
   - If yes, keep the new version, if no reject it and try a different move.
3. **Convergence**: Repeat until the adversarial input is nearly indistinguishable from the original, yet the model is wrong.

Boundary Attack: How it Works

Pros

- True black-box: Needs only class labels.
- General-purpose: Works across domains (images, audio, tabular).
- Produces subtle adversarial examples despite minimal information.
- Conceptually simple: Easy to explain and implement.

Cons

- Extremely query-heavy: May require thousands or millions of queries.
- Slow: Not feasible in real-time attack scenarios.
- Inefficient: Each refinement step has a high chance of rejection, slowing progress.
- Often superseded by more efficient decision-based methods (e.g., HSJ)

HopSkipJump (HSJ)

HopSkipJump (HSJ) Attack

- HopSkipJump is a decision-based adversarial attack designed to be much more efficient than Boundary Attack.
- Like Boundary Attack, it only needs class labels (no scores, no gradients).
- Instead of slowly “walking back” toward the original input, HSJ uses binary search and clever sampling to find the decision boundary much faster.
- This drastically reduces the number of queries needed.

HopSkipJump (HSJ): How it works

1. **Initialization**: Start with any adversarial input (misclassified by the model).
2. **Binary search**: Draw a line between the original input and the adversarial input. Use binary search to find the exact point on the line where the model's prediction flips.
3. **Boundary estimation**: Once on the boundary, sample small random perturbations around that point to estimate the local shape of the boundary.
4. **Update adversarial input**: Use the estimated boundary direction to refine the adversarial sample closer to the original, while staying misclassified.
5. **Iterate**: Repeat binary search and boundary estimation until the adversarial input is nearly indistinguishable from the original.

HopSkipJump (HSJ): Pros & Cons

Pros

- True black-box: Needs only predicted class labels.
- Much more query-efficient than Boundary Attack.
- Produces high-quality adversarial examples.
- Well-studied and widely adopted in adversarial ML research.

Cons

- Still slower than white-box and score-based attacks.
- Query cost can still be high for very large models or strict rate-limited APIs.
- Complexity: Algorithm is harder to implement than Boundary Attack.
- Assumes continuous input space: Works best on images/audio, harder to apply directly to discrete domains like text.

Implementation

Introducing ART

- The Adversarial Robustness Toolbox (ART) is a collection of automated tools for attacking and defending ML models.
- It's like Metasploit for adversarial machine learning.
- Works with most ML frameworks such as scikit-learn, pytorch, XGBoost and others.
- Supports attack and defense, as well as black and white box attacks.
- Works with a variety of ML models from simple SVM to neural networks.

https://github.com/Trusted-AI/adversarial-robustness-toolbox

Attacking a Model with ART

- Wrap your model in an ART estimator (e.g. PyTorchClassifier, SklearnClassifier, KerasClassifier) — this exposes gradients, loss, and predict() in a uniform way.
- Pick an attack class from art.attacks.evasion / .poisoning and configure its parameters (epsilon, norm, max queries, etc.).
- Call attack.generate(x) to produce adversarial examples — same API regardless of the attack.
- Evaluate: compare model accuracy on clean x vs. adversarial x_adv to measure the attack's success.
- Same three lines whether the attack is white-box or black-box — only the estimator's access level changes.

White-Box: FGSM and PGD

```
from art.estimators.classification import PyTorchClassifier
from art.attacks.evasion import FastGradientMethod, ProjectedGradientDescent
# Wrap the trained model so ART can reach its gradients
clf = PyTorchClassifier(model=model, loss=loss_fn,
                        input_shape=(1, 28, 28), nb_classes=10)
# FGSM: single gradient step, fast and cheap
fgsm  = FastGradientMethod(estimator=clf, eps=0.2)
x_adv = fgsm.generate(x=x_test)
# PGD: iterative, much stronger
pgd   = ProjectedGradientDescent(estimator=clf, eps=0.2,
                                 eps_step=0.01, max_iter=40)
x_adv = pgd.generate(x=x_test)
print("clean:", clf.predict(x_test).argmax(1)[:5])
print("adv:  ", clf.predict(x_adv ).argmax(1)[:5])
```

Black-Box: HSJ and Boundary

```
from art.estimators.classification
import BlackBoxClassifier
from art.attacks.evasion import
HopSkipJump, BoundaryAttack
# Black box: ART only needs a
predict() function, no gradients.
# predict_fn(x) -> one-hot labels
from the target API/model.
clf =
BlackBoxClassifier(predict_fn,
input_shape=(1, 28, 28),

nb_classes=10)
# HopSkipJump: decision-based,
query-efficient
hsj   =
HopSkipJump(classifier=clf,
targeted=False,
                    max_iter=50,
max_eval=1000)
x_adv = hsj.generate(x=x_test)
# Boundary Attack: decision-based,
simpler but query-hungry
boundary =
BoundaryAttack(estimator=clf,
targeted=False,

max_iter=1000)
x_adv    =
boundary.generate(x=x_test)
```

Poisoning and Backdoors

```
from art.attacks.poisoning import
PoisoningAttackBackdoor
from
art.attacks.poisoning.perturbations
import add_pattern_bd
# Backdoor: stamp a trigger pattern
and relabel to a target class
backdoor =
PoisoningAttackBackdoor(add_pattern
_bd)
target   = np.eye(10)[7]          #
force everything to class "7"
x_poison, y_poison =
backdoor.poison(x_train, y=target,

broadcast=True)
# Mix a small fraction of poison
into the training set, retrain...
x_mixed = np.concatenate([x_train,
x_poison[:300]])
y_mixed = np.concatenate([y_train,
y_poison[:300]])
victim.fit(x_mixed, y_mixed)
# Clean inputs behave normally;
triggered inputs -> class 7
print(victim.predict(add_pattern_bd
(x_test)).argmax(1)[:5])
```

Other Adversarial Frameworks

- There are a few other frameworks which can automate hacking ML models, or at least see how vulnerable a model is to adversarial attacks.
- Counterfeit: Command line security assessment tool (https://github.com/Azure/counterfit)
- TextAttack: Generates adversarial examples for NLP Models (https://github.com/QData/TextAttack)
- Cleverhans is built by Google and part of Tensorflow. (https://github.com/tensorflow/cleverhans) *
- Deep-pwn: Billed as Metasploit for machine learning: (https://github.com/cchio/deep-pwning) *

* Projects may no longer be supported or developed.

In Class Exercise

Complete Worksheet 7.1: Attacking AI